

Cera Functional Language - 1.0

Isaac Honeyman

July 2, 2026

Abstract

This document outlines the architectural pipeline, implementation phases, and execution mechanics of the Cera custom functional programming language compiler and runtime system.

Contents

1	Core Design Ideas	3
1.1	Philosophy & Paradigm	3
1.2	Architecture & Execution	3
1.3	Lexical Conventions	3
2	Language Syntax	4
2.1	Types	4
2.2	Variable Declarations and Scoping	4
2.3	Operators and Expressions	5
2.4	Control Flow	5
2.5	Functions	6
2.6	Custom Types (Algebraic Data Types)	7
2.7	Error Handling	8
2.8	Deferred Evaluation	8
2.9	Program Entry and I/O	8
2.10	Miscellaneous Syntax	9
2.11	Summary	9
3	Lexical Analysis (Tokenisation)	10
3.1	The Token Structure	10
3.2	Lexical Categories and Regular Expressions	10
3.3	Whitespace and Comment Handling	11
3.4	Import Handling	11
4	Syntax Analysis (Parsing)	12
4.1	Notation Guide	12
4.2	Context-Free Grammar (CFG)	12
4.3	Lexical Ambiguity Resolution	13
4.4	Operator Precedence and Associativity	13
5	Abstract Syntax Tree (AST) Implementation	14
5.1	Node Architecture and Immutability	14
5.2	Pratt Parsing Mechanics	14
5.3	Tree Traversal via Pattern Matching	15

6	Semantic Analysis	15
6.1	Environment and Symbol Resolution	15
6.2	Hindley-Milner Type Inference (Algorithm W)	16
6.3	Generic Instantiation and Polymorphism	16
6.4	Operator and Expression Validation	16
7	Intermediate Representation (IR)	17
7.1	Closure Conversion and Lambda Lifting	17
7.2	A-Normal Form (ANF) Flattening	17
8	Bytecode Emission and Instruction Set Architecture	17
8.1	Instruction Format	17
8.2	The Opcode Categories	18
8.3	The Constant Pool	19
9	The Virtual Machine (VM)	19
10	Planned Updates	19
10.1	Excluded Features	19
10.2	Planned Features	19

1 Core Design Ideas

1.1 Philosophy & Paradigm

- **True Functionality:** The language adheres strictly to pure functional paradigms, enforcing zero side effects (excluding isolated I/O) and strictly immutable state.
- **First-Class Functions:** Functions are treated as fundamental data types, supporting higher-order programming, closures, and localized lambda expressions.
- **Evaluation Strategy:** The language enforces strict, call-by-value evaluation. However, deferred execution and lazy infinite structures can be elegantly synthesized via `unit` thunks (e.g., `unit -> g`).
- **Simplistic, Streamlined Design:** The language minimizes syntactic bloat. Rather than offering a dozen slightly varying keywords to solve a problem, Cera focuses on a highly orthogonal set of fundamental operations that can be composed to solve any complex task.

1.2 Architecture & Execution

- **Tech Stack:** The bytecode compiler, including the lexical, syntactic, and semantic pipelines, is authored in C# to leverage modern pattern-matching and immutable records. The runtime Virtual Machine is written in C for deterministic memory control and high-performance execution.
- **Type System:** Cera utilizes a Hindley-Milner (HM) type inference engine. Global function signatures require explicit typing, establishing a rigid structural boundary, while local variable bindings benefit from seamless HM type inference.
- **Memory Model:** Heap allocations for recursive Algebraic Data Types and closures are managed via a deterministic Reference Counting system.
- **Backwards Compatibility:** The AST and bytecode serialization maintain strict versioning rules to ensure legacy Cera code compiles flawlessly on newer runtime iterations.
- **Build System:** The compilation, testing, and deployment pipeline is orchestrated via a unified Bash scripting ecosystem.

1.3 Lexical Conventions

- **OOP Style Syntax:** While mathematically pure, the surface syntax deliberately mirrors C-family/OOP languages (C#, Java) to maximize readability and reduce the barrier to entry for modern developers.
- **Variables:** Identifiers utilize `camelCase`. Explicit type annotations are encouraged for documentation but can be omitted when contextually obvious to the HM inference engine.
- **Functions:** Function identifiers enforce `camelCase` and strictly require an explicit type signature. This rule applies uniformly across user-defined and compiler-intrinsic functions.
- **Types:** Built-in primitives and user-defined Algebraic Data Types strictly use `lowercase` identifiers, whereas their concrete data constructors require `PascalCase`.

2 Language Syntax

2.1 Types

The language base type set will be extremely minimal, all using lowercase names, with the following types and type 'extensions':

- **int**: 64-bit signed integer (2's complement)
- **float**: 64-bit IEEE 754 floating point number
- **bool**: 1-bit boolean value (true or false)
- **char**: 32-bit Unicode character
- **unit**: A type representing the absence of a meaningful mathematical value, used for side effects, and delayed evaluation. It has exactly one valid value, written as `()`.
- **list**: linked list of values of a single type (OCaml-style)
- **arr**: fixed-length contiguous array, assignable once upon creation, addressable via the built in `get` keyword.
- **tup**: fixed-length tuple of values of potentially different types, note this is not a keyword unlike other extensions, and type definition must be wrapped in brackets to prevent ambiguity.

Note that in the following snippets, we can chain together extensions to create more complex types, as well as the syntax sugar for a char list.

Cera Language

```
def foo() : unit = {  
  var x: int = 5;  
  var y: int list = [1, 2, 3, 4];  
  var z: char list list = [['a', 'b'], ['c', 'd']];  
  var s: char list = "hello world";  
  var w: bool arr = arr[true, false, true];  
  var u: (int * int * int) = (1, 2, 3);  
  var f: int -> int -> int =  
    func (x: int, y: int) : int = x + y;  
  ();  
}
```

2.2 Variable Declarations and Scoping

Variables in Cera are immutable bindings, explicitly or implicitly typed upon creation. Because the language enforces zero mutable state, a variable's value can never be altered after it is bound.

- **Bindings**: Variables are declared using the `var` keyword. If a type annotation is omitted, the compiler utilizes local type inference to determine the type from the evaluated expression.
- **Shadowing**: Cera supports variable shadowing within the same scope. Re-declaring a variable with an existing identifier creates a completely new memory binding that obscures the original identifier for the remainder of the scope, preserving immutability.
- **Strings**: The language does not have a distinct string primitive; syntactic sugar can be used, and strings are evaluated syntactically as a `char list`.

Data Transformation via Shadowing

```
def normalizeVector(x: float, y: float, z: float) :  
(float * float * float) = {  
  
    // 1. Calculate the magnitude  
    var mag: float = (x * x) + (y * y) + (z * z);  
  
    // 2. Safely handle zero-vectors without mutating state  
    var mag: float = (mag == 0.0) ? 1.0 : mag;  
  
    // 3. 'mag' is shadowed; we securely divide by the new 'mag'  
    (x / mag, y / mag, z / mag);  
}
```

2.3 Operators and Expressions

In Cera, all operations are evaluated strictly by default. The language supports standard arithmetic, relational, and logical operators. Because Cera is a strictly typed language, operators do not implicitly cast between types (e.g., an `int` cannot be added directly to a `float` without explicit conversion). This process shall be done by built-in function calls.

- **Arithmetic:** Standard binary operators for numeric types: `+`, `-`, `*`, `/`, and `%` (modulo) (integer and float).
- **Bitwise:** Bitwise operators: `&`, `|`, `^`, `~`, `<<`, and `>>` (integer).
- **Relational:** Comparison operators returning a `bool`: `==`, `!=`, `<`, `>`, `<=`, `>=`. Structural equality is evaluated for composite types like tuples and lists (only `==` and `!=`).
- **Logical:** Boolean operators: `&&` (logical AND), `||` (logical OR), and `!` (logical NOT). These support short-circuit evaluation.
- **Concatenation and Lists:** The `::` operator is used to prepend an element to a list. The use of a `concat` function will concatenate two lists/arrays of the same type.
- **Conditional:** Use of the `?` and `:` operators for inline conditional expressions (ternary operator).

Expressions and Operators

```
def foo() : unit = {  
    var a: int = 10 + 5;  
    var b: float = 3.14 * 2.0;  
    var isValid: bool = (a > 20) && (b < 10.0); // false  
    var sameTuple: bool = (1, 2) == (1, 2); // true  
    var combined: int list = concat([1, 2], (3::[4, 5]));  
    var result: int = isValid ? 100 : a > 20 ? 200 : 300; // 300  
    ();  
}
```

2.4 Control Flow

Because Cera enforces zero mutable state, standard control flow statements are replaced by **control flow expressions**. Every conditional construct evaluates to a return value. To maintain type safety, the compiler strictly enforces that all branches within a single control flow construct evaluate to the exact same type.

- **Expression Blocks:** Standard `if/else` and `else if` blocks that evaluate to the value of their final expression. The `return` keyword is not used; the last evaluated expression in the block is returned implicitly. Note that the final expression of a block does not require a `;` and the inclusion is up to a programmers intuition.
- **Ternary Operator:** The inline `? :` operator is supported for concise, single-line conditional assignments. Chained ternary operators are right-associative.
- **Pattern Matching:** The `switch` construct replaces traditional statement-based switch blocks. It is the primary mechanism for structurally unpacking tuples, lists, and complex types. Branches are defined using the `->` operator.

Pattern Matching a Token Stream

```
def evaluateFlags(flags: char list list, verbose: bool) :
bool = {
  switch (flags) {
    // Base case: stream is empty
    [] -> verbose,

    // Destructure the head and recursive tail
    (h::tl) -> if (h == "v") {
      evaluateFlags(tl, true);
    } else {
      // Ignore unknown flags, continue parsing
      evaluateFlags(tl, verbose);
    }
  }
}
```

2.5 Functions

Functions in Cera are first-class citizens. Because the language enforces zero mutable state, there are no traditional looping constructs (e.g., `for` or `while` loops); all iteration is achieved via recursion.

- **Declarations:** Named functions are declared using the `def` keyword. All parameters and the return type must be explicitly annotated. The body of the function is an expression block, and the final evaluated expression is implicitly returned.
- **Higher-Order Functions:** Functions can be passed as arguments to other functions and returned as values, utilizing the arrow typing syntax (e.g., `int -> int`).
- **Recursion:** Functions in Cera are recursive by default, allowing a function to call itself within its own body without additional keywords.
- **Anonymous Functions (Lambdas):** Lambda functions are defined using the `func` keyword, followed by explicitly typed parameters, a strictly required return type annotation, and either an expression block or a single inline expression.
- **Generic Functions:** Cera supports generic functions by appending type parameters in angle brackets directly after the function name (e.g., `def name<g1, g2>`). These generic types can be used throughout the function signature and body.
- **Scoping and Shadowing:** All named functions (declared via `def`) must reside in the global scope and possess strictly unique identifiers. Function shadowing is not permitted,

and named inner functions are forbidden to maintain a flat AST. Localized closures must be handled via anonymous lambda functions.

- **Function Application:** Cera does not support automatic partial function application (currying). A function call must supply the exact number of arguments specified in its signature. Developers requiring partially applied functions must explicitly construct them using lambdas.

Higher-Order Data Pipelines

```
def sumEvenSquares(lst: int list) : int = {  
  // 1. Filter out the odd numbers  
  var evens = filter(lst, func (x: int): bool = x % 2 == 0);  
  
  // 2. Map the remaining numbers to their squares  
  var squares = map(evens, func (x: int): int = x * x);  
  
  // 3. Fold the list to accumulate the final sum  
  foldLeft(squares, func (acc: int, x: int): int = acc + x, 0);  
}
```

2.6 Custom Types (Algebraic Data Types)

Cera supports the creation of custom Algebraic Data Types (ADTs). To align with the built-in primitive types, user-defined type identifiers are strictly lowercase, while their data constructors must be capitalized (PascalCase). This guarantees the lexer can instantly distinguish between a type signature and a concrete data constructor.

- **Sum Types (Variants):** Defined using the `type` keyword. Constructors are separated by the `|` (OR) operator. If a constructor carries a data payload, the `:` operator is used to specify the underlying type.
- **Recursive Types:** Types can reference themselves within their own definitions, allowing for recursive data structures like trees and streams.
- **Generic Types:** Type parameters are declared using angle brackets (e.g., `<g>`) and passed consistently to recursive calls.

Recursive Immutable Data Structures

```
// A generic Binary Tree definition  
type tree<g> = Leaf | Node : (tree<g> * g * tree<g>);  
  
def insertInteger(t: tree<int>, val: int) : tree<int> = {  
  switch (t) {  
    Leaf -> Node(Leaf, val, Leaf),  
    Node(left, v, right) ->  
      if (val < v) {  
        Node(insertInteger(left, val), v, right) }  
      else if (val > v) {  
        Node(left, v, insertInteger(right, val)) }  
      else { t } // return the unmodified tree  
  }  
}
```

2.7 Error Handling

To maintain absolute mathematical purity and predictable control flow, Cera completely omits `try/catch` exception blocks and the concept of `null` references. Instead, functions that can fail or return no value must make that failure explicit in their type signature using built-in Algebraic Data Types.

- **The Option Type:** Used when a function might legitimately return nothing. It is defined as `type option<g> = None | Some: g`.
- **The Result Type:** Used when a function can fail and needs to return an error message or code. It is defined as `type result<v, e> = Ok: v | Error: e`.

These types force the programmer to handle both the success and failure states using `switch` expressions, guaranteeing that unhandled exceptions cannot crash the VM.

Error Handling via ADTs

```
def divide(a: float, b: float): result<float, string> = {
  if (b == 0.0) {
    Error("Division by zero");
  } else {
    Ok(a / b);
  }
}
```

2.8 Deferred Evaluation

Cera evaluates all expressions strictly (call-by-value) by default. However, to support infinite data structures and deferred execution, one may use unit thunks.

Infinite Fibonacci Stream

```
type stream<g> = Nil | Cons : (g * (unit -> stream<g>));

def fibonacci(a: int, b: int) : stream<int> = {
  // Defer the recursive calculation behind a unit lambda
  var nextElement =
    func (u: unit) : stream<int> = fibonacci(b, a + b);

  Cons(a, nextElement);
}

def fibStream() : stream<int> = {
  fibonacci(0, 1);
}
```

2.9 Program Entry and I/O

Cera enforces zero mutable state and forbids user-defined side effects. However, a program must interact with the host system to be useful. This is achieved through controlled, built-in impure functions (intrinsic) and a strictly defined execution entry point.

- **Impure Intrinsic:** I/O operations (such as reading files or writing to standard output) are provided exclusively as compiler built-ins. Users cannot author their own impure functions from scratch within Cera.

- **The Entry Point:** Program execution begins strictly at the `entry` function. This function serves as the bridge between the host operating system and the pure functional runtime. It takes an array of command-line arguments and must evaluate to an integer representing the system exit code (where 0 indicates success).

Program Entry

```
def entry(args: char list arr): int = {
  // Built-in impure function call
  out("hello world");
  var status: int = 0; // Evaluates to the exit code
  status;
}
```

2.10 Miscellaneous Syntax

- **Comments:** Cera supports single-line comments using the standard `//` syntax. The lexer ignores all text from the `//` token to the end of the line. Block comments can be implemented using `/*` to start and `*/` to end, allowing for multi-line comments that can be nested.
- **Multifile Programs:** Cera takes a C-style approach to simple multifile programs. To use another file, a Cera file must begin with the syntax `import "FileName";`. These are then recursively dealt with (keeping track of imported files to prevent cyclic loops) and are treated as essentially one large file. Note that due to this, types and function identifiers must be unique across all files.

2.11 Summary

To assist in the lexical analysis phase, the following is an exhaustive list of reserved keywords in the Cera language. These identifiers are strictly reserved by the grammar and cannot be used as variable, function, or custom type names.

- **Declaration & Binding:** `var`, `def`, `func`, `type`
- **Control Flow:** `if`, `else`, `switch`, `import`
- **Primitive Types:** `int`, `float`, `bool`, `char`, `list`, `arr`, `unit`
- **Boolean Literals:** `true`, `false`

Note: Identifiers such as `concat`, `out`, and `entry` are considered built-in intrinsic functions rather than reserved lexical keywords. Next is a list of built-in ADTs:

- **Option:** `type option<g> = None | Some: g`
- **Result:** `type result<v, e> = Ok: v | Error: e`

Finally, the list of built-in functions:

- **Get:** `get<g>(target: g arr, index: int) : option<g> (0 based array index).`
- **Arr Length:** `arrLength<g>(array: g arr) : int`
- **Build:** `build<g>(size: int, f: (int -> g)) : g list`
- **Arr Build:** `arrBuild<g>(size: int, f: (int -> g)) : g arr`
- **Concat:** `concat<g>(left: g list, right: g list) : g list`

- **Arr Concat:** `arrConcat<g>(left: g arr, right: g arr) : g arr`
- **Out:** `out(output: char list) : unit`
- **In:** `in() : char list`
- **Read:** `read(path: char list) : result<char list, char list>`
- **Write:** `write(path: char list, content: char list) : result<unit, char list>`
- **Int To Float:** `intToFloat(x: int) : float`
- **Float To Int:** `floatToInt(x : float) : int (truncates)`
- **Char To Int:** `charToInt(c : char) : int`
- **Int To Char:** `intToChar(x : int) : char`
- **Int To Chars:** `intToChars(x : int) : char list`
- **Float To Chars:** `floatToChars(x : float) : char list`
- **Bool To Chars:** `boolToChars(x : bool) : char list`
- **Chars To Int:** `charsToInt(str : char list) : option<int>`
- **Chars To Float:** `charsToFloat(str : char list) : option<float>`
- **Arr To List:** `arrToList<g>(array: g arr) : g list`
- **List To Arr:** `listToArr<g>(lst: g list) : g arr`

3 Lexical Analysis (Tokenisation)

The lexical analysis phase operates as a Deterministic Finite Automaton (DFA), consuming the raw UTF-8 character stream of the source file and segmenting it into a linear sequence of logical `Token` structures. This process strips away non-semantic formatting, preparing the data for syntax analysis.

3.1 The Token Structure

Each emitted token encapsulates three critical pieces of data to ensure both accurate parsing and robust error reporting:

- **Tag (Type):** An enumerated value classifying the token (e.g., `IDENTIFIER`, `INT_LIT`, `DEF_KW`, `PLUS_OP`).
- **Lexeme:** The concrete string of characters that formed the token.
- **Location Metadata:** The line and column index marking the token’s origin, strictly utilized for traceback generation during semantic or syntactic failures.

3.2 Lexical Categories and Regular Expressions

The DFA categorizes lexemes based on the following rules and regular expressions. The scanner employs the “maximal munch” principle, always matching the longest possible valid lexeme (e.g., reading `<=` as a single relational operator rather than a `<` followed by an `=`).

- **Identifiers:** Variables and functions map to `[a-z][a-zA-Z0-9_]*`. Custom type constructors must begin with a capital letter, mapping to `[A-Z][a-zA-Z0-9_]*`.

- **Keywords:** Reserved language keywords (e.g., `var`, `def`, `switch`) are initially scanned as standard identifiers. Before emission, they are checked against a static hash map and re-tagged with their specific keyword enumeration to achieve $O(1)$ lookup times.
- **Literals:**
 - *Integer:* `[0-9]+`
 - *Float:* `[0-9]+\.[0-9]+`
 - *Character:* `'[^']*'`
- **Operators and Punctuation:** Single-character punctuation (e.g., `+`, `*`, `{`, `;`) transitions immediately to an accept state. Potential multi-character operators (e.g., `==`, `->`, `::`) require a single character of lookahead to determine the correct transition.

The following is a full list of tokens used within the lexer:

- **Declaration:** `Var`, `Def`, `Func`, `Type`
- **Control Flow:** `If`, `Else`, `Switch`, `Import`
- **Primitive Types:** `Int`, `Float`, `Bool`, `Char`, `List`, `Arr`, `Unit`, `Wildcard`
- **Literals:** `True`, `False`, `Identifier`, `Constructor`, `IntLiteral`, `FloatLiteral`, `CharLiteral`, `StringLiteral`
- **Flow:** `LBracket`, `RBracket`, `LBRace`, `RBrace`, `LPar`, `RPar`
- **Structural:** `Equal`, `SemiColon`, `Comma`
- **Operators:** `Plus`, `Minus`, `Star`, `Slash`, `Mod`, `BitAnd`, `BitXor`, `BitNot`, `LShift`, `RShift`, `EqualEqual`, `NotEqual`, `Lesser`, `LesserEqual`, `Greater`, `GreaterEqual`, `And`, `Or`, `Not`, `ColonColon`, `Question`, `Colon`, `Pipe`, `Arrow`,
- **Special:** `None`

Note: The `None` token type is simply used for errors, and represents the absence of a token.

3.3 Whitespace and Comment Handling

Whitespace characters (spaces, tabs, carriage returns, and newlines) serve strictly as lexeme delimiters. When the DFA encounters whitespace, it forces the completion and emission of the currently accumulating token. The whitespace itself is then immediately consumed and discarded; no token is generated.

Similarly, single-line (`//`) and block (`/* ... */`) comments transition the scanner into an absorption state, aggressively consuming characters until the termination sequence is reached, emitting nothing to the parser stream.

3.4 Import Handling

As will be seen in the next section (Parsing), the import keyword and its syntax `import "filename"` are not present. This is an intentional choice to prioritise simplicity of our compilation process, and is handled instead in the code via an intermediate `ImportResolver` stage, which searches the top of lexed files for imports, such the entire included codebase can be parsed as one unit.

4 Syntax Analysis (Parsing)

4.1 Notation Guide

The syntax is formally specified using Extended Backus-Naur Form (EBNF). To avoid ambiguity between the meta-syntax and the language’s own lexical tokens, the following typographic conventions are strictly applied:

- $\langle \text{Rule} \rangle$: Angle brackets denote a **non-terminal** production rule.
- **keyword**: Bold, monospace text denotes a **terminal** symbol (a concrete lexical token, keyword, or required punctuation like (or }).
- $::=$: The definition operator, read as “is defined as”.
- $|$: The alternation operator, denoting a choice between derivations (“or”).
- $[\dots]$: Square brackets enclose an **optional** component (zero or one occurrences).
- $(\dots)$: Standard mathematical parentheses denote **logical grouping** of meta-symbols. They are *not* literal characters in the source code.
- x^* : The Kleene star suffix indicates **zero or more** consecutive repetitions of x .
- x^+ : The Kleene plus suffix indicates **one or more** consecutive repetitions of x .

4.2 Context-Free Grammar (CFG)

The formal CFG for Cera is defined in Backus-Naur Form (BNF). To satisfy structural requirements for unambiguous derivations and Top-Down parsing without left recursion, the grammar stratifies expressions and types. The grammar enforces strict functional purity by restricting **var** bindings strictly to local expression blocks.

Are top level fundamental definitions:

```
 $\langle \text{Program} \rangle ::= (\langle \text{FuncDecl} \rangle \mid \langle \text{TypeDecl} \rangle)^*$   
 $\langle \text{FuncDecl} \rangle ::= \mathbf{def\ id} \ [ \langle \text{GenericDecl} \rangle ] \ ( [ \langle \text{Param} \rangle \ (, \langle \text{Param} \rangle)^* ] ) : \langle \text{Type} \rangle = \langle \text{ExprBlock} \rangle$   
 $\langle \text{TypeDecl} \rangle ::= \mathbf{type\ id} \ [ \langle \text{GenericDecl} \rangle ] = \langle \text{ConDecl} \rangle \ ( \mid \langle \text{ConDecl} \rangle )^* ;$   
 $\langle \text{GenericDecl} \rangle ::= < \mathbf{id} \ (, \mathbf{id})^* >$   
 $\langle \text{Type} \rangle ::= \langle \text{SuffixType} \rangle \ [-> \langle \text{Type} \rangle]$   
 $\langle \text{SuffixType} \rangle ::= \langle \text{AtomType} \rangle \ [ \mathbf{list} \mid \mathbf{arr} ]^*$   
 $\langle \text{AtomType} \rangle ::= \langle \text{BaseType} \rangle \mid \langle \text{TupleType} \rangle \mid \langle \text{GenericType} \rangle \mid ( \langle \text{Type} \rangle )$   
 $\langle \text{BaseType} \rangle ::= \mathbf{int} \mid \mathbf{float} \mid \mathbf{bool} \mid \mathbf{char} \mid \mathbf{unit}$   
 $\langle \text{TupleType} \rangle ::= ( \langle \text{Type} \rangle \ ( * \langle \text{Type} \rangle )^+ )$   
 $\langle \text{GenericType} \rangle ::= \mathbf{id} < \langle \text{Type} \rangle \ (, \langle \text{Type} \rangle )^* >$   
 $\langle \text{ConDecl} \rangle ::= \mathbf{con} \ [ : \langle \text{Type} \rangle ]$   
 $\langle \text{ExprBlock} \rangle ::= \{ \langle \text{Stmt} \rangle^* \langle \text{Expr} \rangle [ ; ] \}$   
 $\langle \text{Stmt} \rangle ::= \langle \text{VarDecl} \rangle \mid \langle \text{Expr} \rangle ;$   
 $\langle \text{Param} \rangle ::= \mathbf{id} : \langle \text{Type} \rangle$ 
```

Following are the expression definitions:

```

⟨Expr⟩ ::= ⟨BinOpChain⟩ [? ⟨Expr⟩ : ⟨Expr⟩]
⟨BinOpChain⟩ ::= ⟨UnaryExpr⟩ (⟨BinOp⟩ ⟨UnaryExpr⟩)*
⟨UnaryExpr⟩ ::= ⟨UnOp⟩ ⟨UnaryExpr⟩ | ⟨CallExpr⟩
⟨CallExpr⟩ ::= ⟨Primary⟩ ((⟨Expr⟩(, ⟨Expr⟩)*))
⟨Primary⟩ ::= ⟨Literal⟩ | id | (⟨Expr⟩) | ⟨IfExpr⟩ | ⟨SwitchExpr⟩ | ⟨Lambda⟩
⟨BinOp⟩ ::= + | - | * | / | % | & | | | ^ | << | >> | == | != | < | > | <= | >= | && | || | ::
⟨UnOp⟩ ::= ! | ~ | -
⟨IfExpr⟩ ::= if (⟨Expr⟩) ⟨ExprBlock⟩ (else if (⟨Expr⟩) ⟨ExprBlock⟩)* [else ⟨ExprBlock⟩]
⟨SwitchExpr⟩ ::= switch (⟨Expr⟩) {⟨PatternMatch⟩ (, ⟨PatternMatch⟩)*}
⟨PatternMatch⟩ ::= ⟨Pattern⟩ -> ⟨Expr⟩
⟨Pattern⟩ ::= ⟨Literal⟩ | id | - | (⟨Pattern⟩ (, ⟨Pattern⟩)+) | [[⟨Pattern⟩ (, ⟨Pattern⟩)*]]
| arr [[⟨Pattern⟩ (, ⟨Pattern⟩)*]] | (⟨Pattern⟩ :: ⟨Pattern⟩)
| con[(⟨Pattern⟩ (, ⟨Pattern⟩)*)]
⟨Lambda⟩ ::= func [(⟨Param⟩ (, ⟨Param⟩)*)] : ⟨Type⟩ = (⟨Expr⟩ | ⟨ExprBlock⟩)
⟨Literal⟩ ::= int_lit | float_lit | char_lit | str_lit | true | false | () | ⟨ListLit⟩ | ⟨ArrLit⟩ | ⟨TupleLit⟩
⟨ListLit⟩ ::= [[⟨Expr⟩ (, ⟨Expr⟩)*]]
⟨ArrLit⟩ ::= arr [[⟨Expr⟩ (, ⟨Expr⟩)*]]
⟨TupleLit⟩ ::= (⟨Expr⟩ (, ⟨Expr⟩)+)

```

Note: That **id** and **con** refer to identifier and constructor tokens respectively.

4.3 Lexical Ambiguity Resolution

Because the lexical analyser adheres strictly to the “maximal munch” principle, the character sequence `>>` is always greedily tokenised as a single right-shift operator (`>>`), rather than two distinct `>` tokens. This creates a standard structural collision in $LL(k)$ parsers when evaluating nested generic type closures (e.g., `option<stream<g>>>`).

To resolve this without breaking lexical encapsulation or resorting to a “lexer hack”, the ambiguity is handled entirely within the syntax analyser. When the parser is evaluating a generic type signature and expects a closing bracket `>`, but instead encounters a right-shift `>>` token, it consumes the operator, logically applies the first `>`, and modifies the token stream (or internal parser state) to inject a synthetic `>` token for the subsequent closure evaluation.

4.4 Operator Precedence and Associativity

To eliminate structural ambiguity during the parsing of complex expressions, Cera enforces a strict, formalized hierarchy of operator precedence. The parser utilizes a Top-Down Operator Precedence (Pratt Parsing) architecture, assigning binding powers to individual operator tokens.

A critical architectural departure from legacy C-family languages is Cera’s handling of bitwise operators. Historically, languages like C and Java assign bitwise AND, OR, and XOR a lower precedence than relational comparison operators, leading to non-intuitive evaluations (e.g., `a & b == c` evaluating as `a & (b == c)`). Cera corrects this by enforcing that all bitwise operations bind more tightly than equality and comparison checks, prioritizing mathematical correctness.

Operators are evaluated in the following order, from highest precedence (tightest binding) to lowest precedence:

1. **Call & Grouping:** `()`
2. **Unary Operators:** `!, ~, -`
3. **Multiplicative (Factor):** `*, /, %` (Left-associative)
4. **Additive (Term):** `+, -` (Left-associative)
5. **List Concatenation:** `::` (Right-associative)
6. **Bitwise Shift:** `<<, >>` (Left-associative)
7. **Bitwise AND:** `&` (Left-associative)
8. **Bitwise XOR:** `^` (Left-associative)
9. **Bitwise OR:** `|` (Left-associative)
10. **Comparison:** `<, >, <=, >=` (Left-associative)
11. **Equality:** `==, !=` (Left-associative)
12. **Logical AND:** `&&` (Left-associative, Short-circuiting)
13. **Logical OR:** `||` (Left-associative, Short-circuiting)
14. **Conditional (Ternary):** `? :` (Right-associative)

When operators of identical precedence appear within the same expression, their associativity dictates the order of evaluation. Left-associative operators are evaluated strictly from left to right. The *only* right-associative operators in the Cera language are list concatenation (`::`) and the ternary conditional (`? :`), which evaluate from right to left, natively supporting recursive structural evaluation.

5 Abstract Syntax Tree (AST) Implementation

The translation of the parsed token stream into a logical hierarchy is governed by a purely immutable Abstract Syntax Tree (AST). The C# implementation completely eschews traditional, deep object-oriented inheritance hierarchies in favor of data-centric structures that mirror Cera's own functional paradigms.

5.1 Node Architecture and Immutability

The AST is constructed exclusively using C# `record` types, guaranteeing that all nodes are deeply immutable upon creation. To rigidly categorize nodes and enforce compile-time safety without relying on behavioral inheritance, the compiler utilizes empty marker interfaces: `INodeAST`, `IExprAST`, `IStmtAST`, `ITypeAST`, and `IPatternAST`.

This architecture ensures that structural constraints (such as a `BinaryExpr` strictly requiring a left and right `IExprAST`) are enforced by the type system, while keeping the raw data representations entirely decoupled from compiler operations.

5.2 Pratt Parsing Mechanics

Expression evaluation utilizes a Top-Down Operator Precedence (Pratt) parsing architecture, orchestrated via an `InitializePrattParser()` registry. Rather than relying on deeply nested, rigid recursive descent grammar methods for every mathematical precedence level, the parser assigns semantic parsing delegates to specific token types dynamically:

- **Prefix Parsers:** Tokens that structurally initiate an expression (e.g., literals, identifiers, unary operators, or opening brackets) are mapped to a `PrefixParseFn` delegate.
- **Infix Parsers:** Tokens that operate between existing expressions (e.g., binary operators, the ternary question mark, or the function call parenthesis) are mapped to an `InfixParseFn` delegate, explicitly paired with their operational `Precedence` weighting.

During evaluation, the parser continuously polls the token stream for its assigned binding power. It greedily consumes tokens and recursively invokes their registered delegates for as long as the upcoming token's precedence strictly exceeds the current binding context, naturally producing a correctly ordered AST.

Overall we are implementing a Hybrid LL(1) & Pratt parser.

5.3 Tree Traversal via Pattern Matching

Because the AST nodes are implemented as pure data records devoid of behavioral logic, the compiler deliberately omits the traditional Object-Oriented Visitor pattern (i.e., nodes do not implement `Accept(IVisitor)` methods).

Instead, subsequent compiler phases, such as Semantic Analysis, Type Checking, and Bytecode Emission traverse the AST utilizing modern C# functional pattern matching. By utilizing `switch` expressions over the base marker interfaces (like `IExprAST`), the compiler can cleanly and exhaustively unwrap node structures. This centralizes pipeline logic within the respective compiler phases, avoids interface bloat, and aligns the compiler's codebase with the functional design philosophy of the Cera language itself.

6 Semantic Analysis

Following syntax analysis, the compiler enters the semantic phase. This stage operates over the immutable Abstract Syntax Tree (AST), performing context-sensitive validation, scope resolution, and rigorous type checking to guarantee program safety prior to Intermediate Representation (IR) lowering.

6.1 Environment and Symbol Resolution

Scope in Cera is managed via a hierarchical, purely functional `Environment` architecture. Each local block (e.g., function bodies, lambda expressions, and pattern matching branches) generates a new environment layer linked to its parent.

- **The Scope Chain:** The environment acts as a singly-linked list of hash maps. Each block creates a new environment pointing to its enclosing parent.
- **Symbol Table & Complexity:** Identifiers are mapped to immutable `Symbol` records (e.g., `VarSymbol`, `FuncSymbol`, `TypeSymbol`, `ConstructorSymbol`). Variable lookups operate at $O(1)$ time complexity within the local scope. If a symbol is not found, the traversal up the environment chain operates at $O(K)$ time, where K is the depth of the nested scope.
- **Immutability Guarantee:** Shadowing creates a completely new `VarSymbol` in the localized $O(1)$ hash map rather than mutating the parent environment. This guarantees the language's strict immutability constraint is enforced directly at the compiler architecture level.

6.2 Hindley-Milner Type Inference (Algorithm W)

Cera employs a robust Hindley-Milner (HM) type inference engine based on Damas and Milner’s Algorithm W. While function signatures require explicit type annotations, local variable bindings (`var`) and intermediate expressions rely on HM unification to statically deduce their types.

- **Type Variables:** Unknown types are initially assigned a fresh, sequentially generated `TypeVar` (e.g., T_1, T_2). The algorithm traverses the AST bottom-up, generating type equations and solving them dynamically via unification.
- **Structural Unification:** The `Unify` function mathematically equates expected and actual types. When unifying composite types, the algorithm recursively unpacks the structures. For example, unifying $(\text{int} * T_1)$ with $(T_2 * \text{float})$ successfully resolves T_1 to `float` and T_2 to `int` through parallel structural traversal.
- **Recursive Protection:** This is a crucial safety mechanism. If a user writes an unbounded recursive function, the unification engine might attempt to equate a type variable T_1 with a function signature containing itself (e.g., $T_1 \rightarrow \text{int}$). The explicit “occurs check” halts this process, preventing the compiler from falling into an infinite loop attempting to construct an infinitely sized type graph.
- **Arity and Unit Matching:** Zero-arity functions mathematically evaluate as taking a single `unit` parameter. The syntax `()` evaluates to the structural type `unit`, ensuring complete parity between explicit thunks (`func(u: unit)`) and standard zero-argument functions.

6.3 Generic Instantiation and Polymorphism

Cera supports parametric polymorphism for functions and ADTs. To prevent generic type collisions during recursive function calls or nested data structures, the semantic analyzer utilizes a strict instantiation pipeline.

- **Polytypes vs. Monotypes:** When an Algebraic Data Type (ADT) or generic function is registered in the global environment, its signature is stored as a “polytype” containing unbound generic identifiers.
- **Preventing Variable Capture:** If the compiler unified these polytypes directly during a recursive call, it would permanently bind the global generic parameter to a specific local type, corrupting all future calls.
- **Monomorphization via Substitution:** To solve this, the `Replace` method intercepts the call. It generates a localized mapping of fresh type variables and exhaustively replaces the global parameters across the entire AST node structure, including deep traversals into `TupleType`, `ListType`, `ArrType`, and `FuncType`. This converts the generic polytype into a localized “monotype” safe for HM unification.

6.4 Operator and Expression Validation

To guarantee runtime safety and eliminate algorithmic ambiguity, semantic analysis rigidly enforces operator constraints before unification.

- **Mathematical Operations:** Operators such as `+`, `-`, `*`, and `/` forcefully constrain their operands to numeric base types (`int` or `float`). Ambiguous generic operands defaulting to these operators are strictly bound to `int`.
- **Relational Operations:** Operators such as `<` and `>=` execute numeric validation on their operands but strictly return a unified `bool` type.

- **Ad-Hoc Polymorphism Resolution:** In purely inferred functional systems, overloaded operators cause severe ambiguity when evaluating unannotated lambdas. Because Cera’s grammar strictly mandates explicit type hints for all lambda and function parameters, types are statically bound before operators are evaluated. This entirely sidesteps a classic functional compiler dilemma without relying on external structures like Type Classes.

7 Intermediate Representation (IR)

Following semantic analysis, the verified Abstract Syntax Tree (AST) must be lowered into an Intermediate Representation (IR). Because the target Virtual Machine operates on a linear instruction stream, the hierarchical nature of the AST must be algorithmically flattened, and functional abstractions must be mapped to tangible memory operations.

7.1 Closure Conversion and Lambda Lifting

Cera treats functions as first-class citizens, heavily utilizing anonymous lambdas and closures. However, the underlying C-based Virtual Machine does not natively support nested function declarations. To bridge this gap, the IR phase executes a strict Closure Conversion pass:

- **Free Variable Analysis:** The compiler traverses every lambda expression to identify “free variables”—identifiers used within the lambda that are bound in an enclosing outer scope.
- **Lambda Lifting:** The body of the anonymous lambda is extracted and elevated to the global scope as a discrete, named function (e.g., `__lambda_01`).
- **Closure Allocation:** The original lambda AST node is replaced with an IR instruction to allocate a Closure object on the heap. This object contains a function pointer to the lifted global function, alongside an array of the captured free variables required for its execution.

7.2 A-Normal Form (ANF) Flattening

To simplify bytecode emission, nested complex expressions are transformed into a sequential, linearized format resembling A-Normal Form (ANF). Deeply nested mathematical or functional calls (e.g., `f(g(x))`) are structurally decomposed into sequential atomic assignments. Every intermediate evaluation is bound to a temporary, compiler-generated local variable index. This ensures that when the tree is finally mapped to bytecode, the evaluation stack remains highly predictable, and register/stack slot allocation can be performed in $O(N)$ time.

8 Bytecode Emission and Instruction Set Architecture

The final phase of the C# compiler pipeline is serialization. The flattened Intermediate Representation is traversed to emit binary operational codes (opcodes) defined by Cera’s custom Instruction Set Architecture (ISA).

8.1 Instruction Format

The bytecode is designed as a contiguous array of unsigned 8-bit integers (`uint8_t` in C). The ISA utilizes a variable-length instruction format to optimize memory density:

- **Opcodes:** A single byte representing the operation (e.g., `0x01` for ADD).

- **Operands:** Depending on the opcode, following bytes may represent inline data, such as a 1-byte register index (supporting up to 256 local variables per frame), a 2-byte constant pool index, or a 2-byte relative jump offset.

8.2 The Opcode Categories

To support the pure functional runtime, the ISA is broadly categorized into the following instruction sets:

- **Stack Operations:** Instructions to maneuver data between the local execution frame and the evaluation stack.
 - `LOAD_CONST [index]`: Pushes a literal value from the constant pool onto the stack.
 - `PUSH_INT [byte]`: A fast-path for small integers to save constant pool space.
 - `PUSH_UNIT`: Pushes the `()` singleton to the stack.
 - `LOAD_LOCAL [index]`: Pushes the value of a local variable (by its 1-byte index) onto the stack.
 - `STORE_LOCAL [index]`: Pops the top of the stack and binds it to a local variable slot, adjusting reference counts accordingly.
 - `POP`: Discards the top of the stack.
- **Arithmetic and Logic Unit (ALU):** Standard mathematical operations. These implicitly pop the top two values from the evaluation stack, execute the binary operation, and push the resulting value back.
 - **Math:** `ADD`, `SUB`, `MUL`, `DIV`, `MOD`
 - **Bitwise:** `BIT_AND`, `BIT_OR`, `BIT_XOR`, `BIT_NOT`, `SHL`, `SHR`
 - **Relational:** `EQ`, `NEQ`, `LT`, `GT`, `LTE`, `GTE`
 - **Logical (Unary):** `NOT`, `NEGATE`
- **Control Flow:** Because Cera utilizes expression blocks rather than traditional loops, control flow is handled exclusively via relative jump instructions dictated by conditional evaluations.
 - `JUMP [offset]`: An unconditional forward or backward jump.
 - `JUMP_IF_FALSE [offset]`: Pops the top boolean from the stack; if false, branches execution by the given 16-bit byte offset. This instruction natively powers `if/else` blocks, pattern matching, and short-circuiting for the `&&` and `||` operators.
- **Functions and Closures:**
 - `MAKE_CLOSURE [func_index] [upvalue_count]`: Constructs a closure on the heap using the global function at `func_index`, capturing `upvalue_count` local variables currently on the stack.
 - `CALL [arity]`: Invokes a standard function or closure. Expects the target callable and `arity` arguments to be pre-loaded onto the stack.
 - `CALL_INTRINSIC [intrinsic_id]`: Fast-path invocation for built-in compiler functions (e.g., `out`, `arrBuild`), bypassing the standard call frame allocation overhead.
 - `RETURN`: Pops the final evaluated value of the current frame, tears down the environment (decrementing local reference counts), and returns control to the caller.

- **Algebraic Data Types and Collections:**

- `ALLOC_CON [tag] [arity]`: Allocates a Custom Algebraic Data Type variant on the heap. `tag` identifies the specific constructor, and `arity` determines how many payload values to pop from the stack into the constructor’s memory footprint.
- `MATCH_TAG [tag] [offset]`: Inspects the ADT at the top of the stack. If its tag does not match `tag`, it jumps to `offset` (the next branch of a `switch`).
- `EXTRACT_FIELD [index]`: Extracts the payload field at `index` from the ADT currently at the top of the stack and pushes it.
- `ALLOC_ARRAY [size]`: Allocates a fixed contiguous array on the heap.
- `LIST_CONS`: Pops a head and a list tail, returning a newly heap-allocated list node.

8.3 The Constant Pool

To keep the instruction stream highly compact, large literals (64-bit integers, 64-bit floats, and string constants) are not embedded directly into the bytecode instructions. Instead, they are deduplicated and compiled into an adjacent “Constant Pool” array. The bytecode simply references these values via a 16-bit integer index using the `LOAD_CONST` instruction.

9 The Virtual Machine (VM)

The runtime environment executing the bytecode via a continuous Fetch-Decode-Execute cycle. Define the structure of your virtual register array, evaluation stacks, and runtime memory layouts.

10 Planned Updates

10.1 Excluded Features

Before considering future features for the language, we shall consider previous planned additions, that were decided against for various reasons:

- **Lazy Keyword, Force Pattern:** When the language was first being developed, the idea of a `lazy` keyword was planned, allowing for delayed evaluation thunks similar to unit thunks, but caching the results upon evaluation. This aforementioned similarity, was the primary reason for its exclusion (see design principles), as well as adding a vast, complicated system, that can be replicated without the use of unique keywords and syntax.

10.2 Planned Features

Now we may take an optimistic look into the future of the language:

- **Inline Keyword:** Very similar to its implementation among OOP languages, inline functions will simply insert their body directly in place of its call, rather than requiring an extra frame to be placed onto the stack. While a feature that will be added, omitted for now to reduce initial complexity. e.g. `def inline double(x : float) : float = { 2 * x; }`
- **Top Level Var Declarations:** This can be extremely useful feature, replacing features such as the currently rather clunky format of constants `def pi() : float = 3.14159265358979; ,` and has been omitted from the first version of Cera purely for garbage collector and passing complexity reduction.

- **Switch If Statements:** An embedded way for switch statements to use ifs, similar to OCaml's `when` keyword, as well as the removal of the requirement of braces for if statements.
- **Default Function Values:** Similar to C#, with `param : type = value`.
- **Hidden Keyword:** Basic protection modifier, reduces the scope of a type or function to purely the file it was written in.
- **Inner Named Functions:** As the name states, this functionality is possible with variables and recursive lambda calls, but syntactically ugly, and could be improved.
- **Better Error Messages:** Certain error messages appear vague, missing end of patterns or similar syntax will point to end of file, as well as column number being hard to track.